Subword Tokenization in LLMs

CS115B – Spring 2026
James Pustejovsky

March 31, 2026

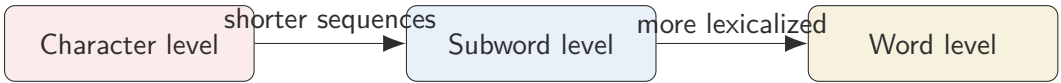
Today

1. Why subwords?
2. Byte Pair Encoding (BPE)
3. From BPE to modern tokenizers
4. SentencePiece
5. tiktoken and byte-level practice
6. Comparisons and design choices
7. Mathematical and systems details
8. Classroom examples and exercises
9. Takeaways

Why do we need subword tokenization at all?

- Word-level vocabularies are brittle: rare words, names, misspellings, compounds, and morphology produce a long tail.
- Character-level models are flexible, but sequences become long and harder to optimize.
- Subword methods seek a middle ground:
  - smaller, reusable units than words;
  - shorter sequences than characters;
  - open-vocabulary behavior without a huge word lexicon.
- In modern LLMs, tokenization affects:
  - context length,
  - training efficiency,
  - inference cost,
  - multilingual robustness,
  - how prompting “feels” to the model.

A basic tradeoff picture



maximal flexibility
but very long sequences

useful compromise
for most neural NLP

compact when seen before
but poor OOV behavior

Guiding idea

Choose a token inventory that balances compression of the input with reuse of recurring internal structure.

The long-tail problem

- Natural language has a Zipfian frequency profile.
- A small number of forms are extremely common.
- A vast number of forms are rare.
- Word-level vocabularies therefore waste capacity:
  - either keep many rare whole words,
  - or replace them with `<unk>`.

Examples

- `electromagnetism`
- `electromagnetic`
- `electromagnetically`
- `electromagnetics`
- `microtokenization`

Observation

These forms share internal structure. A good tokenizer should let the model exploit that reuse.

Formalizing the tokenization objective

Given a string $x = c_1 c_2 \dots c_n$ over a base alphabet Σ, tokenization chooses a segmentation

$$T(x) = (t_1, \dots, t_m) \quad \text{such that} \quad t_1 \circ t_2 \circ \cdots \circ t_m = x,$$

where each t_i belongs to a vocabulary V.

Typical desiderata:

(i) m should be reasonably small

(ii) $|V|$ should be manageable

(iii) V should generalize to unseen words

(iv) T should be efficient and deterministic (or controllably stochastic).

Core tension

Compression of sequences versus robustness to new surface forms.

BPE: the main intuition

- Start from a small base inventory, usually characters plus a word-end marker or boundary convention.
- Count adjacent symbol pairs in the training corpus.
- Merge the most frequent pair into a new symbol.
- Repeat for a fixed number of merges.

Greedy compression view

BPE incrementally builds larger units that capture frequent local co-occurrences.

Important

BPE does not directly optimize linguistic morphology. It optimizes recurring symbol-pair frequency under a greedy merge schedule.

A tiny BPE training example

Suppose the training multiset is:

$$\{\texttt{low}, \texttt{lower}, \texttt{lowest}, \texttt{newer}, \texttt{wider}\}.$$

Represent each word as characters, with end-of-word marker `</w>`:

$$\begin{aligned}
\texttt{low} &\mapsto \texttt{l o w </w>} \\
\texttt{lower} &\mapsto \texttt{l o w e r </w>} \\
\texttt{lowest} &\mapsto \texttt{l o w e s t </w>} \\
\texttt{newer} &\mapsto \texttt{n e w e r </w>} \\
\texttt{wider} &\mapsto \texttt{w i d e r </w>}
\end{aligned}$$

Then count all adjacent pairs and greedily merge the most frequent one.

BPE merges on the toy example

One possible merge sequence is:

$$(\texttt{e},\texttt{r}) \to \texttt{er}, \quad (\texttt{l},\texttt{o}) \to \texttt{lo}, \quad (\texttt{lo},\texttt{w}) \to \texttt{low}, \quad (\texttt{e},\texttt{s}) \to \texttt{es}, \quad (\texttt{es},\texttt{t}) \to \texttt{est}.$$

After a few merges we might get:

```
low </w>
low er </w>
low est </w>
n e w er </w>
w i d er </w>
```

What happened?

Frequent fragments such as `low`, `er`, and `est` became reusable vocabulary items.

BPE algorithm sketch

Given corpus $\mathcal{D}$ represented as sequences over a current vocabulary V:

1. Initialize V with base symbols.
2. For each adjacent pair (a, b), compute count

$$f(a,b) = \sum_{x \in \mathcal{D}} \#_{(a,b)}(x).$$

3. Choose the most frequent pair

$$(a^*, b^*) = \arg\max_{(a,b)} f(a,b).$$

4. Add merged symbol ab to V.
5. Replace every occurrence of (a^*, b^*) by the new token a^*b^*.
6. Repeat for K merges.

Hyperparameter

K = number of merges. Larger K produces a larger vocabulary and usually shorter token sequences.

How BPE tokenizes a new word

Suppose our learned subwords include:

$$\{\texttt{un}, \texttt{happi}, \texttt{ness}, \texttt{happy}, \texttt{er}, \texttt{est}\}.$$

For a new input like `unhappiness`, the tokenizer greedily applies known merges or longest matching subpieces, yielding something like

$$\texttt{un happi ness}.$$

Practical effect

Even if `unhappiness` never appeared as a whole word in training, the model can still encode it compositionally.

BPE as sequence compression

Let $L_{\text{chars}}(x)$ be the character length of string x, and let $L_{\text{BPE}}(x)$ be the number of BPE tokens after segmentation.

A useful rough intuition is:

$$L_{\text{chars}}(x) \geq L_{\text{BPE}}(x) \geq L_{\text{words}}(x)$$

for many common inputs, though not universally.

- BPE compresses repeated orthographic patterns.
- It does not guarantee morphologically correct analyses.
- It is data dependent: the same surface form may segment differently under different corpora and vocabulary sizes.

Strengths and weaknesses of BPE

Strengths

- Simple and fast
- Good open-vocabulary behavior
- Effective for rare words
- Strong empirical history in NMT and LLM pipelines

Weaknesses

- Greedy and local
- Segments are not necessarily linguistic morphemes
- Sensitive to preprocessing conventions
- Different merge budgets can change behavior a lot

Class exercise: hand-run three BPE merges

Training corpus:

$$\{\texttt{play}, \texttt{player}, \texttt{played}, \texttt{playing}\}$$

represented with `</w>` boundaries.

1. Count all adjacent symbol pairs.
2. Perform the top merge.
3. Recount and perform two more merges.
4. Predict how `playful` will segment afterward.

Goal

See directly how local frequency builds reusable pieces like `play`, `er`, `ed`, `ing`.

Why plain BPE was not the end of the story

As neural NLP matured, practitioners wanted more than simple merge rules:

- language independence,
- training directly from raw text,
- better treatment of whitespace,
- probabilistic segmentation options,
- byte-level robustness,
- industrial-strength speed.

This is where libraries such as SentencePiece and tiktoken became especially important.

What is SentencePiece?

- SentencePiece is a tokenizer and detokenizer framework designed to train directly from raw text.
- It supports at least two major segmentation families:
  - BPE-style models,
  - unigram language-model tokenization.
- It treats whitespace as a normal symbol, often rendered with a visible marker such as ␣ or the metaspace symbol .

Key idea

Do not assume the input has already been pre-tokenized into words.

Why raw-text training matters

Traditional tokenization pipelines often assume:

$$\text{raw text} \rightarrow \text{word tokenizer} \rightarrow \text{subword tokenizer}.$$

SentencePiece instead aims for:

$$\text{raw text} \rightarrow \text{subword model directly}.$$

Advantages:

- fewer language-specific heuristics,
- more reproducible preprocessing,
- easier use with languages lacking simple whitespace-separated word boundaries,
- easier multilingual pipelines.

SentencePiece and the whitespace symbol

SentencePiece often escapes whitespace into a learned symbol such as .
For example:

`"New York is big"`

might become roughly

`New York is big.`

Why this is useful

Detokenization becomes easier: concatenating tokens and then converting back to spaces recovers the surface string systematically.

SentencePiece BPE versus unigram LM

SentencePiece BPE

- similar spirit to classical BPE
- merge-driven
- deterministic once trained
- often efficient and strong

SentencePiece unigram LM

- start from a large candidate inventory
- assign probabilities to pieces
- prune pieces that hurt corpus likelihood least
- choose the best segmentation probabilistically

Pedagogical point

SentencePiece is not just one algorithm. It is a framework in which BPE is one option.

Unigram LM tokenization: the basic math

Suppose a string x can be segmented in multiple ways:

$$x = t_1 \circ \cdots \circ t_m.$$

Under a unigram model, assign each piece $t \in V$ a probability $p(t)$, and score a segmentation by

$$P(t_1, \ldots, t_m) = \prod_{i=1}^{m} p(t_i).$$

Equivalently, maximize log-probability:

$$\arg\max_{(t_1,\ldots,t_m)} \sum_{i=1}^{m} \log p(t_i).$$

Interpretation

Unlike greedy BPE merges, this treats tokenization as an optimization problem over candidate segmentations.

A tiny unigram LM example

Suppose the vocabulary includes:

$$\texttt{un},\ \texttt{happy},\ \texttt{happi},\ \texttt{ness},\ \texttt{unhappy},\ \texttt{unhappiness}$$

with probabilities:

token	un	happy	happi	ness	unhappy	unhappiness
$p(t)$	0.08	0.12	0.05	0.10	0.06	0.02

Then

$$P(\texttt{un}, \texttt{happi}, \texttt{ness}) = 0.08 \cdot 0.05 \cdot 0.10 = 0.0004,$$

while

$$P(\texttt{unhappy}, \texttt{ness}) = 0.06 \cdot 0.10 = 0.006.$$

So the model prefers the second segmentation.

Subword regularization and BPE-dropout

SentencePiece also supports stochasticity during training-time tokenization.

- Rather than always using one deterministic segmentation, sample among plausible segmentations.
- This can make downstream models less brittle.
- Intuition: expose the model to multiple subword analyses of the same string.

Why this can help

Tokenization itself is an inductive bias. Controlled noise in that bias can act like data augmentation.

SentencePiece strengths and cautions

Strengths

- raw-text training
- multilingual-friendly
- deterministic or stochastic segmentation
- convenient whitespace handling

Cautions

- vocabulary design still matters
- segmentation can still look unintuitive
- token boundaries remain model-specific

Exercise: compare BPE and unigram LM intuitions

For the word `internationalization`, discuss which method is more naturally described by each statement:

1. "merge the most frequent adjacent pieces repeatedly"
2. "choose the best segmentation under a probabilistic lexicon"
3. "sample alternative segmentations during training"

Then ask:

- Which method gives you a cleaner story about optimization?
- Which method gives you a cleaner story about engineering simplicity?

What is tiktoken?

- `tiktoken` is OpenAI's fast tokenizer library.
- It is used to estimate or reproduce tokenization for OpenAI model encodings.
- It is based on BPE-style tokenization, with model-specific encodings such as `cl100k_base` and `o200k_base`.

Why students should care

This is the tokenizer family you touch when counting tokens, pricing prompts, budgeting context windows, or inspecting how prompts are segmented for current OpenAI models.

Byte-level tokenization intuition

A practical tokenizer often wants to avoid catastrophic unknown symbols.

One strategy is to ensure every input string can be reduced to bytes, then learned BPE merges operate over byte-derived symbols.

Engineering payoff

Any Unicode string can be represented. You do not need an `<unk>` mechanism in the old word-level sense.

Tradeoff

Fallback to byte-oriented pieces can increase token counts for unusual text, emoji, or scripts that are poorly compressed by the learned vocabulary.

Why token counts matter in practice

- Context limits are measured in tokens, not words.
- API costs are metered by tokens.
- Prompt templates that look short to humans may be long to the model.
- Structured text (JSON, code, tables) tokenizes very differently from ordinary prose.

$$\text{cost} \propto \#\text{input tokens} + \#\text{output tokens}.$$

Operational lesson

Tokenization is not just preprocessing. It is a systems concern.

Basic tiktoken usage in Python

```
import tiktoken

enc = tiktoken.get_encoding("cl100k_base")
text = "Tokenization matters for LLMs."
ids = enc.encode(text)
print(ids)
print(len(ids))
print([enc.decode([i]) for i in ids])
```

What this gives you

- token IDs,
- token count,
- one-token-at-a-time inspection.

Model-specific encoding selection

```
import tiktoken

enc = tiktoken.encoding_for_model("gpt-4o-mini")
prompt = "Summarize the following article in 5 bullets."
num_tokens = len(enc.encode(prompt))
print(num_tokens)
```

Caution

Different models may use different encodings. Never assume token counts transfer exactly across tokenizers.

A qualitative tokenization example

These strings may have very different token counts:

1. ordinary English prose
2. camelCase variable names
3. Python code
4. JSON with punctuation-heavy structure
5. emoji-rich chat text

Discussion prompt

Why might `"def compute_attention(q, k, v):"` tokenize differently from `"The cat sat on the mat."`?

tiktoken in the larger conceptual picture



Takeaway

Conceptually, tiktoken is still in the BPE family,
but engineered for byte-level coverage and production-scale speed.

Exercise: which string has more tokens?

For each pair, predict which side yields more tokens under a modern BPE tokenizer:

1. `running` vs. `runnnnnnnning`
2. `Theorem 2.1` vs. $\forall x \exists y$
3. `New York City` vs. `NYC`
4. plain prose vs. pretty-printed JSON

Then justify your answer in terms of reuse, punctuation, rarity, and byte fallback.

BPE, SentencePiece, tiktoken: a first comparison

Method	Core idea	Good at	Watch out for
Classical BPE	greedy pair merges	simple, effective subword compression	preprocessing assumptions
SentencePiece BPE	BPE from raw text	multilingual and reproducible pipelines	still segmentation-specific
SentencePiece unigram	maximize segmentation under token probabilities	flexible probabilistic tokenization	conceptually richer, slightly less elementary
tiktoken	fast byte-level BPE-style tokenizer for model encodings	token counting, production APIs, robust coverage	model-specific behavior

Vocabulary size versus sequence length

Let $|V|$ be vocabulary size and let $m(x)$ be token length of string x.

As a rough design trend:

- increasing $|V|$ tends to reduce average $m(x)$,
- but also increases the embedding/output softmax inventory,
- and may overfit to surface forms with poor reuse.

$$\text{Tokenizer design} \approx \text{balance}(\text{compact sequences},\ \text{manageable lexicon},\ \text{generalization}).$$

Historical note

This tradeoff was central in NMT and remains central in LLM engineering.

What tokenizers get wrong semantically

- Subword boundaries are usually orthographic or statistical, not semantic.
- A token like `tion` or `ing` may look morphologically meaningful, but this is not guaranteed.
- Conversely, a morphologically meaningful unit may be split in a bizarre way because the corpus statistics favored another pattern.

Important for linguists

Do not confuse a useful compression scheme with a theory of morphology or lexical semantics.

Tokenization and multilingual NLP

- Languages differ in morphology, writing system, and whitespace behavior.
- Subword tokenization is often essential for agglutinative, compounding, or script-diverse settings.
- Raw-text approaches and byte-level coverage improve robustness, but they do not eliminate all language bias.

Discussion question

Why might a tokenizer designed mostly on English web text behave less elegantly on Turkish, Finnish, Japanese, or mixed-script social media text?

When tokenization changes model behavior

Different tokenizers can change:

- the number of context positions consumed,
- how often certain fragments recur,
- how easily the model learns names, code, numbers, or markup,
- how prompting heuristics transfer across models.

Practical lesson

A tokenizer is part of the model architecture, not a trivial front-end utility.

Worked comparison: one word, three perspectives

Take the input `tokenization`.

Possible analyses:

- character-level: `t o k e n i z a t i o n`
- BPE-style: `token ization`
- another BPE model: `tok en ization`
- unigram LM: choose the segmentation with highest token-probability product

Moral

There is no uniquely correct tokenization independent of the tokenizer's learned vocabulary and objective.

Tokenization as shortest path / dynamic programming

For probabilistic segmentation methods, a string x with positions $0, 1, \ldots, n$ can be viewed as a DAG:

- node i = character boundary position,
- edge (i, j) exists if substring $x_{i:j}$ is a valid token.

Assign edge weight

$$w(i, j) = -\log p(x_{i:j}).$$

Then best tokenization is the shortest path from 0 to n.

Why this matters

This gives a crisp algorithmic account of unigram tokenization and related segmentation problems.

Expected token length and efficiency

If X is a random input string distribution, then one useful statistic is

$$\mathbb{E}[m(X)]$$

where $m(X)$ is the number of tokens after segmentation.

Why do we care?

- attention cost grows with sequence length,
- context windows are finite,
- throughput depends on token count.

Simplified systems fact

All else equal, fewer tokens usually means cheaper and faster transformer processing.

But bigger vocabularies are not free

A larger vocabulary affects at least two major matrices in a standard LM:

- the input embedding table,
- the output projection / softmax layer.

If hidden dimension is d and vocabulary size is $|V|$, these components are on the order of

$$|V| \times d.$$

Tokenizer design is architecture design

Smaller sequences and larger vocabularies push the cost in opposite directions.

Special tokens and control structure

Modern tokenizers also include special tokens for things like:

- beginning/end of sequence,
- padding,
- unknown or fallback behavior,
- role/control markers in chat formatting,
- tool or modality delimiters.

Teaching point

When students inspect token counts, they should remember that model inputs often include hidden or template-level structure beyond the obvious user text.

Common misconceptions

1. "A token is basically a word." — No.
2. "One tokenizer's counts transfer to another model." — Not reliably.
3. "Subword pieces are true morphemes." — Often false.
4. "Tokenization is solved." — Still an active design space.

Board exercise: segment these items qualitatively

Without running code, propose plausible subword splits for:

1. `misunderstanding`
2. `electroencephalography`
3. `tokenization-aware`
4. `ChatGPT-4o-mini`
5. `printf("hello\n");`

Then discuss which pieces are likely learned because they are frequent, and which pieces probably survive only by character/byte fallback.

Mini lab: inspect tokenization in Python

1. Choose 5 strings: prose, code, math, emoji, and a long technical word.
2. Encode them with a tokenizer.
3. Record:
   - token count,
   - token IDs,
   - decoded individual tokens.
4. Compare the counts and explain the differences.

Good report question

Which kinds of text are compressed most effectively by the tokenizer you tested?

Challenge problem: build the BPE objective yourself

Suppose a corpus contains strings $x^{(1)}, \ldots, x^{(N)}$.

Ask the students:

- If you merge pair (a, b) into ab, how much does the corpus length shrink?
- Why is the greedy choice closely tied to the highest-frequency adjacent pair?
- Can you imagine a globally better merge schedule than the greedy one?

Pedagogical payoff

This connects tokenization to approximation algorithms and local-versus-global optimization.

Prompt-engineering consequence

Two prompts with nearly identical meanings can have noticeably different token counts:

- short words versus technical jargon,
- compressed prose versus verbose JSON scaffolding,
- repeated delimiters or markup.

In-class discussion

When does rewriting a prompt reduce token cost without changing the task?

Code and markup are special cases

Code often has:

- repeated keywords,
- punctuation-rich syntax,
- identifiers with reusable fragments,
- indentation and formatting artifacts.

Markup and JSON similarly combine repeated structure with rare field names.

Implication

A tokenizer that looks efficient on prose can behave quite differently on code-heavy or schema-heavy workloads.

A compact summary table for students

Question	BPE answer	SentencePiece answer	tiktoken answer
What is learned?	merge rules / merged units	BPE or unigram subword inventory from raw text	model-specific byte-level BPE-style encoding
How is whitespace handled?	often external preprocessing	explicit symbolization like	encoding-specific handling
Why use it?	rare-word handling, simplicity	raw-text multilingual pipelines	practical token counting and robust production use
Main limitation	greedy local merges	still tokenizer-dependent boundaries	counts vary by model encoding

The big picture

1. Subword tokenization is a compromise between words and characters.
2. BPE builds reusable pieces by greedy pair merges.
3. SentencePiece generalizes the engineering story and also supports unigram LM tokenization.
4. tiktoken brings fast, practical, model-specific tokenization into everyday LLM use.
5. Tokenization shapes efficiency, robustness, and even model behavior.

What I want you to remember

Three key lessons

1. Tokenizers are learned inductive biases.
2. Their units are statistical artifacts, not guaranteed linguistic atoms.
3. In LLM work, tokenization is both a mathematical design choice and a systems constraint.

References



Rico Sennrich, Barry Haddow, and Alexandra Birch.

2016.

Neural Machine Translation of Rare Words with Subword Units.

ACL 2016.



Taku Kudo and John Richardson.

2018.

SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing.

EMNLP System Demonstrations 2018.



OpenAI.

tiktoken repository and tokenizer documentation.

OpenAI Cookbook and tokenizer documentation.